HOMEWORK

TecFest CySec Week 02



Ngurah Acaryanandha Putra

26053091134

Cyber Security

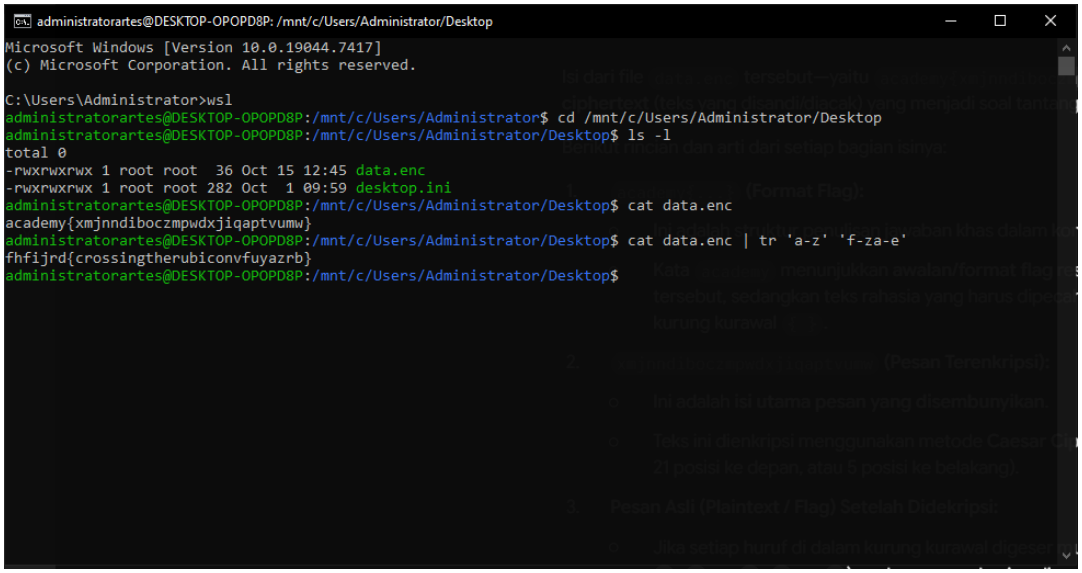
1. Contents

Ngurah Acaryanandha Putra 26053091134 1
1. Tugas General 3
1.1 Caesar 3
1.2 Shift Registers 3
1.3 hashcrack 5
1.4 Mod 26 6
1.5 13 7
1.6 Binary Digits 8
1.7 Inspect HTML 9
Daftar Pustaka 11

1. Tugas General

1.1 Caesar

Jadi pada challange berjudul Caesar ini kita ditugaskan untuk melakukan Decrypt pada file data.enc, ketika saya buka filenya menggunakan command cat data.enc terlihat bahwa isinya adalah flag dari challange ini sendiri. Jadi setelah di analisis teks yang berada pada dalam kurung kurawa tersebut adalah teks yang telah dienkripsi menggunakan metode Caesar Cipher yaitu metode enkripsi yang menggeser abjad sejauh 21 posisi kedepan, atau lima posisi kebelakang, setelah mengetahui hal tersebut saya menggunakan command linux yaitu cat data.enc yang berungsi untuk membaca lagi file tersebut namun bedanya saya menambahkan tr ‘a-z’ ‘f-za-e’ yang berfungsi untuk mengganti setiap karakter abjad berdasarkan pemetaan yang ditentukan jadi abjad normal yaitu ‘a-z’ diubah urutannya menjadi 5 kebelakang jadi f – z dan a-e. Setelah di enter didapatkanlah flag dari challange Caesar ini, dibawah ini disajikan tampilan terminal saya ketika mengerjakan challange ini dan sesuai dengan command yang telah saya ucapkan di awal.



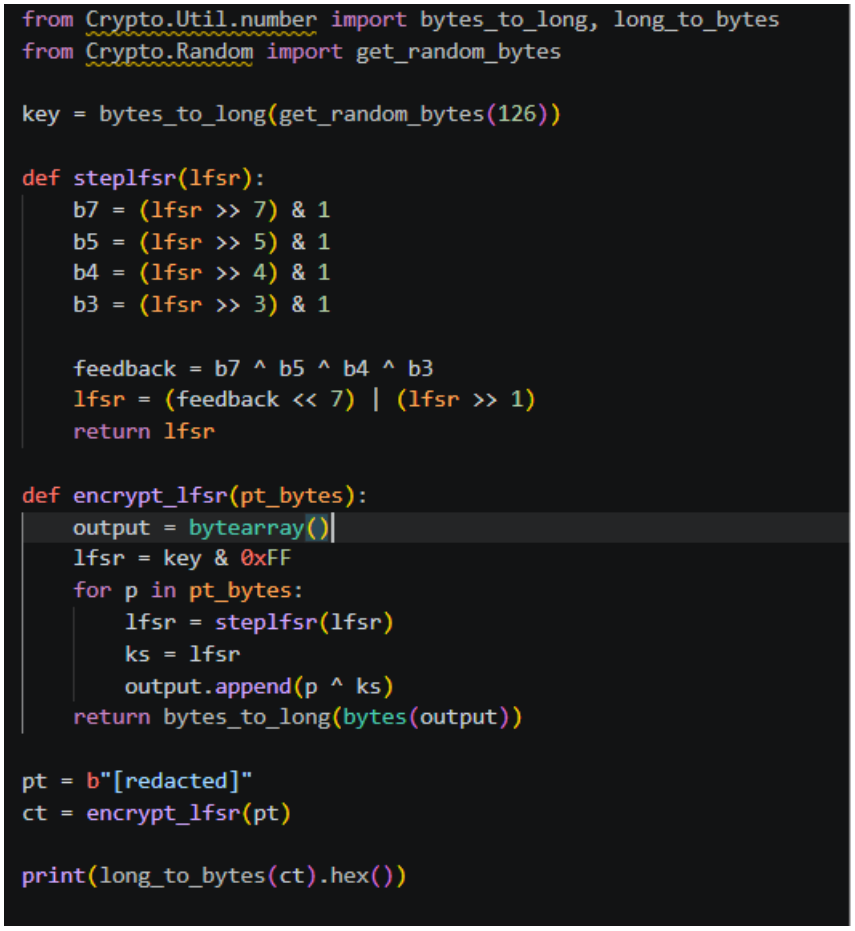
Gambar 1.1 Terminal

Jika diperhatikan awalnya ketika data.enc di baca ternyata awalnya sudah terbentuk academy jadi ketika abjadnya di geser yang awalnya academy jadi fhfijrd, namun dalam melakukan submit pada cylab kita tetap menggunakan academy. Jadi pada challange Caesar ini, flagnya adalah

```
academy{crossingtherubiconvfuyazrb}
```

1.2 Shift Registers

Jadi pada challange berjudul shift registers ini diberikan dua file yaitu chall.py dan output, pertama – tama saya buka file chall.py filenya berisikan kode pemrograman yang akan saya tampilkan pada gambar di bawah ini



```python
from Crypto.Util.number import bytes_to_long, long_to_bytes
from Crypto.Random import get_random_bytes

key = bytes_to_long(get_random_bytes(126))

def steplfsr(lfsr):
    b7 = (lfsr >> 7) & 1
    b5 = (lfsr >> 5) & 1
    b4 = (lfsr >> 4) & 1
    b3 = (lfsr >> 3) & 1

    feedback = b7 ^ b5 ^ b4 ^ b3
    lfsr = (feedback << 7) | (lfsr >> 1)
    return lfsr

def encrypt_lfsr(pt_bytes):
    output = bytearray()
    lfsr = key & 0xFF
    for p in pt_bytes:
        lfsr = steplfsr(lfsr)
        ks = lfsr
        output.append(p ^ ks)
    return bytes_to_long(bytes(output))

pt = b"[redacted]"
ct = encrypt_lfsr(pt)

print(long_to_bytes(ct).hex())
```

Gambar 1.2 kode pemrograman shall.py

Jadi setelah di analisis di bagian awal kode ini bikin kunci sepanjang 126 bit, namun jika dilihat ketika fungsi encrypt_lfsr dijalankan operasi key & 0xFF itu hanya mengambil 8 bit paling bawah dari key, artinya 125 bit sisa dari key itu dibuang. Oleh karena itu kemungkinan nilai awal lfsr itu cuma dari 0 – 255 dan kita ketahui cylab ini flagnya pasti di awali academy jadi kita tinggal cek mana hasil dekripsi nilai lfsr yang di mulai dari kata academy{. Jadi untuk mendapatkan flagnya saya membuat file solusi.py yang berisikan kode programnya dan dibawah ini disajikan kode program solusi.pynya

```python
def steplfsr(lfsr):
    b7 = (lfsr >> 7) & 1
    b5 = (lfsr >> 5) & 1
    b4 = (lfsr >> 4) & 1
    b3 = (lfsr >> 3) & 1

    feedback = b7 ^ b5 ^ b4 ^ b3
    lfsr = (feedback << 7) | (lfsr >> 1)
    return lfsr

ct_hex = "30cbb50e507774fdafd01e0bfdbc38d5eadf92991cdd347064e37444e517d661cea9"
ct_bytes = bytes.fromhex(ct_hex)

for seed_awal in range(256):
    lfsr = seed_awal
    hasil_dekripsi = bytearray()

    for c in ct_bytes:
        lfsr = steplfsr(lfsr)
        ks = lfsr
        hasil_dekripsi.append(c ^ ks)

    if hasil_dekripsi.startswith(b"academy{"):
        print("Seed awal LFSR ", seed_awal)
        print(hasil_dekripsi.decode())
        break
```

Gambar 1.3 kode program solusi.py

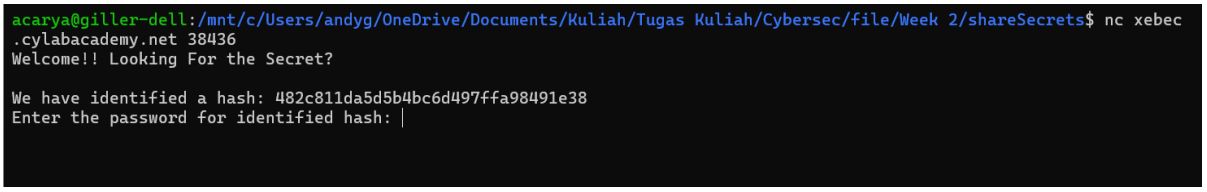
Jadi pada kode program solusi.py ini saya mengambil fungsi steplfsr dari file chall.py di awal yang berfungsi untuk mengecek bit pada angka lfsrnya, kemudian karena fungsi ini matematis dan pasti, urutan ks yang dihasilkan bakal sama persis dengan yang di pakai saat enkripsi kalau kita menggunakan nilai lfsr awal yang benar, jadi disini dimanfaatkan sifat dasar XOR(^). Kemudian saya membuat variabel baru bernama ct_hex yang menyimpan teks heksadesimal hasil enkripsi pada file output.txt, kemudian deretan huruf hex tadi saya rubah menjadi kumpulan bit mentah sehingga bisa di olah satu per satu, dan proses terakhir yaitu brute force, saya membuat perulanga untuk mencoba semua angka dari 0 – 255 sebagai tebakan awal dan disimpan ke variabel lfsr dan juga membuat variabel kosong bertipe bytearray untuk menampung hasil dari dekripsinya. Kemudian setiap kali mau medekripsi satu bit, nilai lfsr akan terupdate menggunakan fungsi steplfsr yang sudah kita siapkan tadi. Nilai lfsr yang baru ini dijadikan sebagai keystream (ks) untuk meng-XOR byte ciphertext (c), lalu hasil dekripsinya dimasukkan ke dalam variabel hasil_dekripsi. Proses ini terus diulang untuk setiap byte, dan di akhir perulangan kita tinggal melakukan pengecekan dengan fungsi .startswith(b"academy{"). Jika hasil dekripsi diawali dengan format flag academy{, berarti nilai tebakan seed awal kita benar! Program akan mencetak nilai seed beserta string flag aslinya, lalu menghentikan perulangan dengan perintah break. Maka flag dari challange ini adalah

```
academy{l1n3ar_f33dback_sh1ft_r3g}
```

1.3 hashcrack

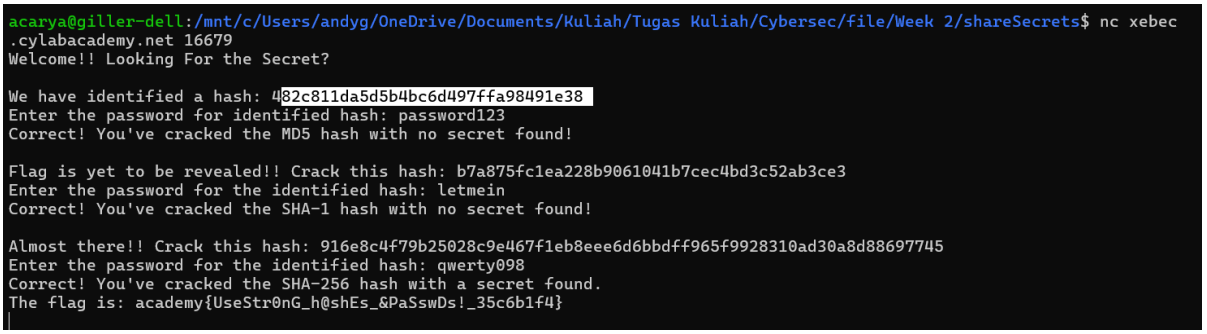
Pada challange berjudul hascrack ini saya diberikan instance berupa netcat yang mengarahkan saya ke bagian soalnya. Challange ini memberikan nilai hash dengan maksud menyutuh kita untuk mencari teks asli dari hash tersebut yang berupa password untuk

mendapatkan flagnya dibawah ini disajikan gambar yang menampilkan terminal saya ketika mengakses netcatnya.



Gambar 1.4 Terminal

Jadi pada challange ini saya menggunakan situs CrackStation untuk mendapatkan teks asli alias passwordnya, dan pada challange ini diberikan total tiga hash yang harus dijawab, dan baru mendapatkan flagnya, dibawah ini disajikan gambar terminal saya yang telah mendapatkan ketiga password dari challange ini.



Gambar 1.5 Terminal

Jadi pada challange hashcrack ini setelah menjawab ketiga hasnya, saya mendapatkan flagnya yaitu

```
academy{UseStr0nG_h@shEs_&PaSswDs!_35c6b1f4}
```

1.4 Mod 26

Pada challange berjudul Mod 26 ini, diberikan suatu file txt, yang berisikan teks alias flag yang telah di encode menggunakan ROT13, jadi pada challange ini supaya tidak menggunakan situs terus, saya membuat file solusi.py dengan kode pemrograman seperti gambar di bawah ini

```python
import codecs

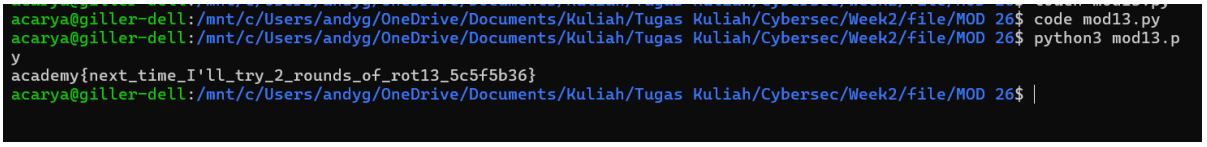
ciphertext = "npnqrzl{arkg_gvzr_V'yy_gel_2_ebhaqf_bs_ebg13_5p5s5o36}"


flag = codecs.decode(ciphertext, 'rot_13')

print(flag)
```

Gambar 1.6 kode pemrograman solusi.py

Jadi pada kode pemrograman di atas saya membuat variabel ciphertext yang berisikan teks yang telah di encode dari file values.txt di awal, kemudian saya membuat variabel baru lagi bernama flag yang menggunakan fungsi codecs.decode() dengan menggunakan aturan rot_13 untuk menggeser setiap huruf pada variabel ciphertext di awal sebanyak 13 langkah, dan terakhir saya print flagnya, dan di bawah ini disajikan gambar yang menampilkan solusi.py ini dijalankan.



Gambar 1.7 terminal

Jadi setelah saya jalankan mod13.py alias kode pemrograman yang telah di buat di awal dan ternyata bisa dan flagnya dapat ditemukan. Jadi flag pada challange Mod 26 ini adalah

```
academy{next_time_I'll_try_2_rounds_of_rot13_5c5f5b36}
```

1.5 13

Jadi pada challange berjudul 13 ini sebenarnya cara kerjanya sama persis seperti challange sebelumnya yang berjudul mod26, diberikan teks alias flagnya yang telah di encoding dengan enkripsi ROT13 jadi kita tinggal menambahkan teks pada challange 13 ini ke dalam kode pemrograman di awal yang telah di buat pada challange mod26.

```python
import codecs

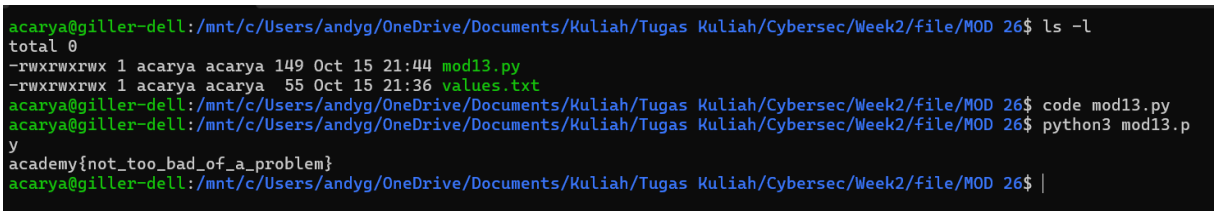
ciphertext = "npnqrzl{abg_gbb_onq_bs_n_ceboyrz}"


flag = codecs.decode(ciphertext, 'rot_13')

print(flag)
```

Gambar 1.8 kode pemrograman solusi

Jadi saya tinggal mengganti isi dari variabel ciphertextnya pada kode pemrograman yang telah di buat pada challange mod26 sehingga ketika kode ini di run akan menghasilkan flag dari challange 13, di bawah ini disajikan tampilan termina ketika menjalankan kode pemrogramannya.



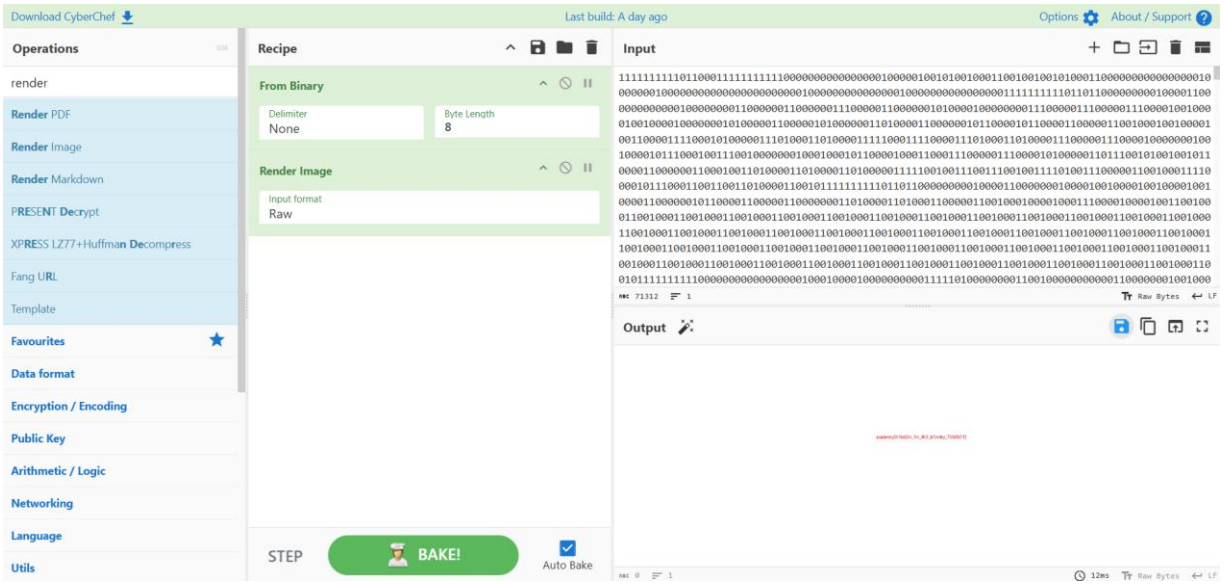
Gambar 1.9 terminal

Sehingga setelah di jalankan akan menampilkan flag dari challange berjudul 13 ini, maka flagnya adalah

```
academy{not_too_bad_of_a_problem}
```

1.6 Binary Digits

Kita akan pindah divisi ke forensic dengan challange berjudul Binary Digits, pada challange ini kita diberikan satu file yaitu digits.bin dan ketika saya buka filenya dengan command cat digits.bin isinya adalah angka biner yang sangat banyak. Jadi menyelesaikan challange ini saya menggunakan situs cyberchef dengan resep from binary dengan delimiter none dan juga render image, dibawah ini ditampilkan gambar yang menyajikan tampilan cyberchef



Gambar 1.10 cyberchef

Jadi pada situs ini setelah di masukkan angka biner yang didapatkan dari membuka file digits.bin di awal dihasilkan suatu output gambar yang jika di perhatikan itu berisikan flagnya, dan setelah di download gambarnya saya tampilkan di bawah

Gambar 1.11 output cyberchef

Jika diperhatikan lebih jelas, maka dapat dilihat flagnya dari gambar tersebut. Maka flag challange ini adalah

```
academy{h1dd3n_1n_th3_b1n4ry_75fdfb71}
```

1.7 Inspect HTML

Jadi pada challange inspect HTML ini diberikan link website yang mengarah ke pada wesbite challangenya, challange ini gampang kita tinggal inspect web tersebut dan menganalisa secara teliti, dan hasil analisa saya mendapatkan flagnya yang disembunyikan, dibawah ini disajikan gambar yang menampilkan tampilan web beserta inspect yang mengarah kepada flagnya

Gambar 1.12 inspect website



Bisa dilihat secara jelas pada bagian kiri terdapat flag yang di sembunyikan. Maka flag pada challange inspect HTML ini adalah

```
academy{1n5p3t0r_0f_h7ml_a2b543be}
```

Daftar Pustaka

Cyber, S. (2023). *picoCTF Writeups: Strings It*. Medium. https://medium.com/@sobatistacyber/picoctf-writeups-strings-it-0e77401efb72

Die.net. (n.d.-a). *nc(1) - Linux Manual Page*. https://linux.die.net/man/1/nc
Die.net. (n.d.-b). *ssh(1) - Linux Manual Page*. https://linux.die.net/man/1/ssh
picoCTF. (n.d.). *picoCTF Primer*. https://primer.picoctf.org/

GCHQ. (n.d.). *CyberChef* [Aplikasi web]. GitHub Pages. Diakses pada 28 September 2026, dari https://gchq.github.io/CyberChef/

DupliChecker. (n.d.). *ASCII to text* [Aplikasi web]. Diakses pada 28 September 2026, dar